

EXP6:CHECKSUM CALCULATION WIRESHARK

RAVI BALA M.V

Register Number: 24BCE1419

Faculty Name: Subbulakshmi

Lab Slot: L55 + L56

Course Name: Computer Networks Lab

Course Code: BCSE308L

Checksum METHOD1: UDP

No.	Time	Source	Destination	Protocol	Length	This is an ACK to the segment in frame	The RIT to ACK the segment was	Info
34	0.019398	2404:a800:6:51:face...	2401:4900:4df2:f787...	UDP	1298			443 → 58153 Len=1232
35	0.019398	2404:a800:6:51:face...	2401:4900:4df2:f787...	UDP	1298			443 → 58153 Len=1232
36	0.019398	2404:a800:6:51:face...	2401:4900:4df2:f787...	UDP	1298			443 → 58153 Len=1232
37	0.019398	2404:a800:6:51:face...	2401:4900:4df2:f787...	UDP	1298			443 → 58153 Len=1232
38	0.019398	2404:a800:6:51:face...	2401:4900:4df2:f787...	UDP	1298			443 → 58153 Len=1232
39	0.019398	2404:a800:6:51:face...	2401:4900:4df2:f787...	UDP	1298			443 → 58153 Len=1232
40	0.019398	2404:a800:6:51:face...	2401:4900:4df2:f787...	UDP	1298			443 → 58153 Len=1232
41	0.019398	2404:a800:6:51:face...	2401:4900:4df2:f787...	UDP	1298			443 → 58153 Len=1232
42	0.019398	2404:a800:6:51:face...	2401:4900:4df2:f787...	UDP	1298			443 → 58153 Len=1232
43	0.019398	2404:a800:6:51:face...	2401:4900:4df2:f787...	UDP	1298			443 → 58153 Len=1232
44	0.019398	2404:a800:6:51:face...	2401:4900:4df2:f787...	UDP	1298			443 → 58153 Len=1232
45	0.019398	2404:a800:6:51:face...	2401:4900:4df2:f787...	UDP	1298			443 → 58153 Len=1232

Frame 55: Packet, 1298 bytes on wire (10384 bits), 1298 bytes captured (10384 bits) on interface 0
Ethernet II, Src: b2:52:e9:42:44:96 (b2:52:e9:42:44:96), Dst: Intel_06:4e:74 (dc:45:46:06:4e:74)
802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 0
Internet Protocol Version 6, Src: 2404:a800:6:51:face:b00c:3333:7020, Dst: 2401:4900:4df2:f787:21e2:8755:b195:f681
User Datagram Protocol, Src Port: 443, Dst Port: 58153
Source Port: 443
Destination Port: 58153
Length: 1240
Checksum: 0xb0416 [unverified]
[Checksum Status: Unverified]
[Stream index: 0]
[Stream Packet Number: 54]
[Timestamps]
UDP payload (1232 bytes)
Data (1232 bytes)
Data [...]: 4309e1bc57aa7979983f7718ab6080697e41545dac3592370610ba659ee90645974bc0ecf9fe8e69544224d7d82de70863558c7c63c7430fb59332a91ea5dc97801093e5ad3a1
[Length: 1232]

0040 04 16 43 09 e1 bc 57 aa 79 79 98 3f 77 18 ab 60 80
0050 80 66 97 e4 15 45 da c3 59 23 70 61 0b a6 59 ee
0060 90 64 59 74 bc 0e cf 9f e8 e6 95 d4 22 4d 7d 82
0070 de 70 86 35 58 c7 c6 3c 74 30 fb 59 33 2a 91 ea
0080 5d c9 78 01 09 3e 5a d3 ab 91 c7 41 b9 a6 db dc
0090 14 61 a3 87 4e d0 04 00 db 24 91 e4 21 75 27 05
00a0 fc c3 cb 58 ba 51 52 e4 45 24 5d 12 fd a0 bc 23
00b0 7d 60 74 7f 53 93 80 bb d5 c6 57 b1 32 ee 44 4e
00c0 d6 84 b6 ab 55 01 fd 8f 0e 9e ad 01 41 7d 03 85
00d0 c6 df 78 08 8e 2c 3b 88 43 82 c6 f5 97 fb 05 c3
00e0 93 a3 db a0 db 9e ae a3 3c 89 5c 5a 67 ac ee 45
00f0 1b 79 c3 70 f5 d9 67 b2 70 83 9d a6 f2 89 c4 54
0100 b4 ea 57 b0 cb dc fb c3 d0 41 22 51 5e 0c 3c 67
0110 08 5b 39 c3 45 f8 91 ec ad 75 2b 62 7d 52 6d 3e
0120 3d 4a 5a cb 3f 6a 3f 6c 1b cd 77 58 f7 07 34 b4
0130 75 1b 85 d6 92 8e 7b 74 72 e0 a6 bc 6d d1 92 3e
0140 a7 18 24 20 d7 9c a5 c2 e7 64 09 c7 6a fd 98 ft
0150 6c ba 5c 78 32 ff 20 6d 19 49 2a 13 32 47 a0 02
0160 9f 70 53 bc e2 0b 21 0c 54 a2 af 66 c0 de 00 75
0170 ed b7 95 17 1e 79 35 1e 2c 77 db 4c 08 4c f0 ca
0180 9d 7a 71 a1 8f e8 9a e2 9e ed f7 af 9f 6e 7c 5b
0190 f6 2b ad 57 1b 4c ff 84 3a a6 47 7c f5 74 89 ac
01a0 f3 3c e6 60 2a 83 40 60 ba 9b e9 aa 86 85 45 b1
01b0 f1 2c dd cd 4c 7d 0d b2 79 0e db 17 ae a6 a6 94
01c0 9a b1 0c 7a bf 1c ee 41 3d 28 c4 8e e3 03 08 cd
01d0 27 a8 00 b0 98 1d dc 00 4c 9b 5d 7d 6e 14 eb 7c
01e0 b1 e1 c9 40 57 18 d9 19 16 88 cd f0 7d 4c 6f b7
01f0 ea 6e b5 e7 a3 d9 d6 db 6a 7e 41 a0 d3 46 fd 5b
0200 e5 d9 08 8d a0 44 8a 02 cf e5 2b a4 fc c7 9f d5

Checksum METHOD2: SCAPY

```
from scapy.all import rdpcap, IP

try:

    packets = rdpcap('capture.pcap')

    print(f"Successfully loaded {len(packets)} packets.")

    for i, pkt in enumerate(packets[:5]): # Checking first 5 packets
        if pkt.haslayer(IP):
            orig_chk = pkt[IP].chksum
            del pkt[IP].chksum
            pkt = pkt.__class__(bytes(pkt))
            new_chk = pkt[IP].chksum

            status = "PASS" if orig_chk == new_chk else "FAIL"
            print(f"Pkt {i} | Original: {hex(orig_chk)} | Calculated: {hex(new_chk)} | Result: {status}")
except FileNotFoundError:
    print("Error: 'capture.pcap' not found. Please capture traffic in Wireshark first!")
```

```
root@Ravi-PC:~# nano verify_checksum.py
root@Ravi-PC:~# python3 verify_checksum.py
Successfully loaded 1607 packets.
```

CODE:

```
GNU nano 7.2
verify_checksum.py

from scapy.all import rdpcap, IP

try:

    packets = rdpcap('capture.pcap')

    print(f"Successfully loaded {len(packets)} packets.")

    for i, pkt in enumerate(packets[:5]): # Checking first 5 packets

        if pkt.haslayer(IP):

            orig_chk = pkt[IP].chksum

            del pkt[IP].chksum

            pkt = pkt.__class__(bytes(pkt))

            new_chk = pkt[IP].chksum

            status = "PASS" if orig_chk == new_chk else "FAIL"

            print(f"Pkt {i} | Original: {hex(orig_chk)} | Calculated: {hex(new_chk)} | Result: {status}")

except FileNotFoundError:

    print("Error: 'capture.pcap' not found. Please capture traffic in Wireshark first!")
```